04_testing_freshness

6. Testing Strategy

Helios is an autonomous diagnosis engine: the agents never write SQL, they compose governed metrics that resolve to marts. If a mart is silently wrong, the Critic cannot catch it (the SQL is "valid"), the reconcile guardrail can pass (both sides share the bug), and a confident, well-cited, *wrong* Decision Brief ships. Tests are therefore not hygiene - they are the only thing standing between a transform bug and a fabricated root cause. We run a six-layer pyramid, widest (cheapest, most numerous) at the base:

Everything runs under `dbt build`, never `dbt run` then `dbt test` separately - `build` interleaves model and test execution in DAG order so a failed test on `int_ga4__sessionized` aborts before the poisoned data ever reaches `fct_funnel`. Running them separately would materialize the whole graph on bad upstream data and only discover it afterward.

6.1 Generic built-in tests

The four built-ins anchor every model's `schema.yml`. PK uniqueness and not-null on join/grain keys are non-negotiable; `accepted_values` pins the closed vocabularies that the semantic layer depends on; `relationships` enforces the layered DAG's referential integrity.

```yaml
# models/marts/core/_core__models.yml
models:
  - name: fct_funnel
    description: "{{ doc('fct_funnel') }}"
    config:
      contract: {enforced: true}     # column types frozen for the semantic layer
    columns:
      - name: session_key
        data_type: string
        tests:
          - unique
          - not_null
      - name: channel_group
        data_type: string
        tests:
          - not_null
          - accepted_values:
              values: ['Direct','Organic Search','Paid Search','Display',
                       'Paid Social','Organic Social','Email','Affiliates',
                       'Referral','Other']
              config: {severity: error}
          - relationships:
              to: ref('dim_channels')
```

```
            field: channel_group
      - name: device_category
        tests:
          - accepted_values: {values: ['desktop','mobile','tablet','smart tv']}
      - name: session_revenue
        tests:
          - not_null:
              config: {where: "reached_purchase"}   # only purchasers must carry revenue
```

`channel_group` is tested with `accepted_values` at the *mart*, not just at `channel_group_case()`, because a macro change plus a stale incremental partition can drift the two apart - we want the failure at the table the agents read.

6.2 Package tests: dbt_utils + dbt_expectations

`packages.yml` pins `dbt-utils`, `dbt-expectations`, and `dbt-project-evaluator` (the latter as a CI lint that fails on un-tested/un-documented models, models reaching across layers, or `fct_` / `dim_` without a PK test). The expectations packages give us range, expression, mutual-exclusivity, and recency assertions that the built-ins lack:

```
- name: fct_daily_funnel
  columns:
    - name: sessions
      tests:
        - dbt_utils.accepted_range: {min_value: 0, inclusive: true}
    - name: conversion_rate          # not stored, but tested as an expression
  tests:
    # funnel monotonicity at the additive grain (counts, not flags)
    - dbt_utils.expression_is_true:
        expression: "purchasing_sessions ≤ begin_checkout_sessions"
    - dbt_utils.expression_is_true:
        expression: "begin_checkout_sessions ≤ add_to_cart_sessions"
    - dbt_utils.expression_is_true:
        expression: "add_to_cart_sessions ≤ view_item_sessions"
    - dbt_utils.expression_is_true:
        expression: "view_item_sessions ≤ sessions"
    - dbt_expectations.expect_column_pair_values_A_to_be_greater_than_B:
        column_A: revenue
        column_B: 0
        or_equal: true
    # production-only: each day must have a fresh row (no-op on static sample, see §7)
    - dbt_utils.recency:
        datepart: day
        field: event_date
        interval: 2
        config:
          severity: "{{ 'error' if target.name == 'prod' else 'warn' }}"

- name: fct_funnel
  tests:
    # a session cannot be both new and returning user in the same row
    - dbt_utils.mutually_exclusive_ranges:
        lower_bound_column: 0
        upper_bound_column: 0   # placeholder; see is_new_user expression test below
    - dbt_utils.expression_is_true:
        expression: "not (is_new_user and ga_session_number > 1)"
```

6.3 Singular tests (full SQL)

Singular tests are bespoke SELECTs in `tests/`; any returned row is a failure. These two encode Helios invariants too cross-cutting for a column test.

```
-- tests/assert_funnel_monotonicity.sql
-- The reached_* flags are MAX-DOWNSTREAM by construction; any session that
```

```
-- violates the chain means the monotonic roll-up logic broke.  Returns the
-- offending sessions (0 rows = pass).
select
    session_key,
    reached_view_item, reached_add_to_cart, reached_begin_checkout,
    reached_add_shipping_info, reached_add_payment_info, reached_purchase
from {{ ref('fct_funnel') }}
where reached_purchase           > reached_add_payment_info
   or reached_add_payment_info   > reached_add_shipping_info
   or reached_add_shipping_info  > reached_begin_checkout
   or reached_begin_checkout     > reached_add_to_cart
   or reached_add_to_cart        > reached_view_item
```

```
-- tests/assert_session_conversion_rate_bounds.sql
-- Session→purchase conversion on this GA4 sample sits ~1–4%.  A rate of 0,
-- a rate >100% (impossible), or a wild swing means sessionization or the
-- funnel flags broke.  Bounds are intentionally generous to avoid false
-- alarms on Black Friday / holiday peaks within the window.
with daily as (
    select
        event_date,
        sum(purchasing_sessions) as purchasers,
        sum(sessions)            as sessions
    from {{ ref('fct_daily_funnel') }}
    group by 1
)
select
    event_date,
    purchasers,
    sessions,
    safe_divide(purchasers, sessions) as conv_rate
from daily
where sessions > 0
  and (
        safe_divide(purchasers, sessions) > 1.0       -- impossible
     or safe_divide(purchasers, sessions) < 0.0005    -- effectively zero → broken
     or safe_divide(purchasers, sessions) > 0.20      -- implausibly high → broken
  )
```

`assert_session_conversion_rate_bounds` is also a soft data-quality canary: it is the dbt analogue of eval scenario `06_no_anomaly_control` (a flat funnel that must NOT trip a finding).

6.4 Custom generic test: test_revenue_reconciles

Money is the load-bearing number in every Decision Brief, so we ship a reusable generic test that reconciles any revenue column on any model back to the raw GA4 `purchase_revenue_in_usd` within a tolerance. It lives in `tests/generic/` and is referenced like a built-in.

```
-- tests/generic/test_revenue_reconciles.sql
{% test test_revenue_reconciles(model, column_name,
                                source_relation=none,
                                source_column='purchase_revenue_in_usd',
                                tolerance=0.005) %}

{# Sum the mart's revenue and the canonical raw revenue, compare relative drift. #}
{%- set src = source_relation or source('src_ga4', 'events') -%}

with mart_total as (
    select coalesce(sum({{ column_name }}), 0) as amt
    from {{ model }}
),
raw_total as (
    select coalesce(sum({{ source_column }}), 0) as amt
    from {{ src }}
```

```
    where event_name = 'purchase'
),
recon as (
    select
        mart_total.amt as mart_amt,
        raw_total.amt  as raw_amt,
        abs(mart_total.amt - raw_total.amt)
            / nullif(raw_total.amt, 0) as rel_drift
    from mart_total cross join raw_total
)
select *
from recon
where rel_drift > {{ tolerance }}      -- any row returned ⇒ drift exceeds 0.5%
   or (raw_amt = 0 and mart_amt <> 0)  -- mart invented revenue

{% endtest %}
```

```
  - name: fct_orders
    columns:
      - name: gross_revenue
        tests:
          - test_revenue_reconciles:
              tolerance: 0.005
              config: {severity: error}
  - name: fct_funnel
    columns:
      - name: session_revenue
        tests:
          - test_revenue_reconciles
```

This is the in-warehouse twin of `warehouse-mcp.reconcile` (§6.6): same 0.5% tolerance, but run at build time so a reconciliation failure blocks the deploy rather than withholding a finding at runtime.

6.5 Unit tests (dbt 1.8+) for the keystone transforms

The KEYSTONE models - `int_ga4__sessionized` and `int_ga4__funnel_steps` - encode logic that **fails silently**: a wrong `MD5` concat order still produces a valid-looking key, a non-monotonic flag still produces a number. Data tests catch these only if a violating row happens to exist in today's data. Unit tests catch them on fixed input, every build, before any real data flows. **Write these first; they are golden-value tests.**

```
# models/intermediate/_int__unit_tests.yml
unit_tests:
  - name: test_sessionize_key_construction
    description: session_key = TO_HEX(MD5(user_pseudo_id - ga_session_id)); deterministic.
    model: int_ga4__sessionized
    given:
      - input: ref('stg_ga4__events')
        rows:
          - {user_pseudo_id: 'u1', ga_session_id: 100, event_name: 'session_start',
             event_timestamp: 1, session_engaged: '1', source: 'google', medium: 'cpc',
             has_gclid: true,  device_category: 'mobile', country: 'US'}
          - {user_pseudo_id: 'u1', ga_session_id: 100, event_name: 'page_view',
             event_timestamp: 2, source: 'google', medium: 'cpc', has_gclid: true}
          - {user_pseudo_id: 'u2', ga_session_id: 100, event_name: 'session_start',
             event_timestamp: 3, source: '(direct)', medium: '(none)', has_gclid: false}
    expect:
      rows:
        # u1+100 collapses 2 events → 1 session; channel from has_gclid+cpc = Paid Search
        - {session_key: "{{ '%s' | format('') }}", user_pseudo_id: 'u1',
           channel_group: 'Paid Search', engaged_session: true}
        - {user_pseudo_id: 'u2', channel_group: 'Direct', engaged_session: false}

  - name: test_reached_flags_are_max_downstream
```

```yaml
    description: a session that only fired 'purchase' must back-fill all upstream reached_* flags.
    model: int_ga4__funnel_steps
    given:
      - input: ref('stg_ga4__events')
        rows:
          - {session_key: 's1', event_name: 'purchase',        purchase_revenue_in_usd: 50.0}
          - {session_key: 's2', event_name: 'add_to_cart',     purchase_revenue_in_usd: null}
          - {session_key: 's3', event_name: 'view_item',       purchase_revenue_in_usd: null}
    expect:
      rows:
        # s1 purchased → EVERY upstream flag is true (monotonic), revenue carried
        - {session_key: 's1', reached_view_item: true, reached_add_to_cart: true,
           reached_begin_checkout: true, reached_add_shipping_info: true,
           reached_add_payment_info: true, reached_purchase: true, session_revenue: 50.0}
        # s2 only added to cart → downstream flags false, upstream true
        - {session_key: 's2', reached_view_item: true, reached_add_to_cart: true,
           reached_begin_checkout: false, reached_purchase: false, session_revenue: 0.0}
        - {session_key: 's3', reached_view_item: true, reached_add_to_cart: false,
           reached_purchase: false, session_revenue: 0.0}

  - name: test_decomposition_inputs_additive
    description: fct_daily_funnel step counts must satisfy sessions≥view≥cart≥ ... ≥purchase
                 so mix/rate decomposition denominators are well-formed.
    model: fct_daily_funnel
    given:
      - input: ref('fct_funnel')
        rows:
          - {session_key: 'a', event_date: '2021-01-01', channel_group: 'Email',
             device_category: 'desktop', country: 'US', is_new_user: true,
             reached_view_item: true, reached_add_to_cart: true, reached_purchase: true,
             session_revenue: 30.0}
          - {session_key: 'b', event_date: '2021-01-01', channel_group: 'Email',
             device_category: 'desktop', country: 'US', is_new_user: true,
             reached_view_item: true, reached_add_to_cart: false, reached_purchase: false,
             session_revenue: 0.0}
    expect:
      rows:
        - {event_date: '2021-01-01', channel_group: 'Email', device_category: 'desktop',
           country: 'US', is_new_user: true, sessions: 2, view_item_sessions: 2,
           add_to_cart_sessions: 1, purchasing_sessions: 1, revenue: 30.0}
```

These three pin the exact behaviors the agents' decomposition (`mix = Σ Δw_i·r_i(t0)` , `rate = Σ w_i(t0)·Δr_i`) depends on. If a refactor breaks sessionization or monotonicity, the unit test fails on synthetic input in <1s, long before the eval gate or production.

6.6 Reconciliation tests vs warehouse-mcp.reconcile

Beyond build-time `test_revenue_reconciles` , CI runs a runtime parity check: for each canonical metric/grain, compare the dbt mart total against `warehouse-mcp.reconcile(metric, grain)` (the independent canonical query the agents trust). Drift must be ≤ 0.5%.

```bash
# ci/reconcile.sh  -- runs after `dbt build`, fails the job on drift
python -m helios.eval.reconcile_check \
  --pairs "revenue:fct_orders,transactions:fct_orders,sessions:fct_funnel,
           conversion_rate:fct_funnel,revenue:fct_daily_funnel" \
  --tolerance 0.005 \
  --fail-on-drift
```

The check guards against the failure mode where both the mart and a hand metric carry the *same* bug: `reconcile` is computed from raw `src_ga4` by a path that does NOT share Helios's macros, so a macro bug surfaces as drift.

6.7 Integration test: the 50-scenario eval harness

The top of the pyramid is the offline benchmark in `eval/scenarios/` (categories `01_single_segment_rate` … `07_data_quality`, ~50 labeled scenarios). It is the **integration test**: it runs the full Monitor→Decompose→Diagnose→Critic→Prescribe chain against fixtures with a known injected root cause and grades the emitted diagnosis. CI promotes it to a **required check** with two hard gates:

- **Root-cause accuracy ≥ 85%** (the Diagnose agent names the correct injected cause; naive baseline ≤ 45%).
- **Hallucination rate = 0** (no column/metric in any emitted SQL is absent from the semantic registry / GA4 schema).

```
# .github/workflows/ci.yml (excerpt)
  eval-gate:
    needs: [dbt-build]
    steps:
      - run: python -m helios.eval.run --suite all --report eval_report.json
      - run: |
          python -m helios.eval.gate eval_report.json \
            --min-root-cause 0.85 \
            --max-hallucination 0.0   # hard zero
```

Crucially, the **data-quality scenarios (`07_data_quality`) double as data-quality assertions**: each injects a defect (null `transaction_id`, duplicated session, revenue that fails to reconcile, a channel outside the 10 groups) and asserts that Helios *detects and withholds* rather than reports. They are the runtime mirror of §6.1–6.4's build-time tests - if a generic/singular test is ever weakened, the matching `07_data_quality` scenario starts failing, so the two layers backstop each other.

6.8 Severity, store_failures, selection, and CI mechanics

Mechanism	Helios policy
`severity: error`	All PK uniqueness/not-null, `test_revenue_reconciles`, channel `accepted_values`, both singular tests, all unit tests. Blocks the build.
`severity: warn`	Recency on the static sample (§7), wide range checks, freshness on non-prod targets.
`store_failures: true`	Set on the two singular tests + `test_revenue_reconciles` so failing rows land in `helios_eval.dbt_test__audit_*` for triage instead of vanishing.
Tags	`tags: ['keystone']` on sessionize/funnel/reconcile tests; `tags: ['freshness']`, `tags: ['contract']`. Selectable via `dbt build -s tag:keystone`.
`--store-failures-map`	Failures persisted only in CI/prod, not dev, to keep dev cheap.

Severity is environment-aware via `{{ 'error' if target.name == 'prod' else 'warn' }}` so freshness/recency never block a dev iteration on the static sample but hard-fail prod.

CI runs in two tiers:

1. **Slim CI on PRs** - `dbt build -s state:modified+ --defer --state ./prod-manifest`, deferring unbuilt upstreams to the prod manifest so only changed models + descendants build and test. This is the fast path.
2. **Full nightly** - `dbt build` (everything) + `ci/reconcile.sh` + the eval gate.

```
# PR check: build only what changed, defer the rest to prod
dbt build --select state:modified+ \
          --defer --state ./prod-manifest \
          --fail-fast
# nightly + the required eval gate
dbt build && ./ci/reconcile.sh && python -m helios.eval.gate ...
```

Order of authoring (and of trust): **keystone golden-value tests first** (unit + singular), because those bugs are invisible to humans and to the Critic; then reconciliation; then the eval gate as the final, holistic backstop.

---

7. Freshness Strategy

Freshness answers one question for the agents: *is the data recent enough to diagnose against?* A Decision Brief built on a stale or half-loaded partition is worse than no brief - it looks authoritative and is silently wrong. So freshness gates the build, and a stale source aborts rather than producing degraded output.

The honesty up front. Our source is `bigquery-public-data.ga4_obfuscated_sample_ecommerce`, a **static, historical** export covering 2020-11-01 … 2021-01-31. It does not update. *Real* source freshness on it is meaningless - the newest shard is permanently years old. We therefore design the **production** strategy for a live daily GA4 export and make every freshness check **degrade gracefully to a no-op / informational signal** on the static sample, gated on `target.name`.

7.1 Source freshness (production)

GA4's BigQuery export lands `events_YYYYMMDD` daily, completed each morning UTC (intraday `events_intraday_*` streaming is a Phase-3 item, explicitly out of scope here). We declare freshness on the source with `loaded_at_field` derived from the event timestamp.

```yaml
# models/staging/_src_ga4__sources.yml
sources:
  - name: src_ga4
    database: "{{ var('ga4_project', 'bigquery-public-data') }}"
    schema: ga4_obfuscated_sample_ecommerce
    # PRODUCTION freshness; made informational on the static sample (see config below)
    freshness:
      warn_after:  {count: 36, period: hour}
      error_after: {count: 48, period: hour}
    loaded_at_field: >
      TIMESTAMP_MICROS(MAX(event_timestamp))
    tables:
      - name: events
        identifier: "events_*"
        # prune date-shards; never scan the full history for a freshness probe
        loaded_at_field: "TIMESTAMP_MICROS(event_timestamp)"
        freshness:
          warn_after:  {count: 36, period: hour}
          error_after: {count: 48, period: hour}
          filter: "_TABLE_SUFFIX ≥ FORMAT_DATE('%Y%m%d', DATE_SUB(CURRENT_DATE(), INTERVAL 5 DAY))"
```

The 36h warn / 48h error thresholds tolerate one missed daily load (e.g. a weekend export hiccup) as a *warning* but treat two consecutive misses as an *error*. The `filter` on `_TABLE_SUFFIX` is essential: a freshness probe must never trigger a full-table scan across the entire shard history - it prunes to the last 5 days so the check costs cents.

Freshness gates the build. CI runs the source check *before* `dbt build`; a hard failure aborts the run.

```bash
# ci/freshness_gate.sh  (production target only)
if [ "$DBT_TARGET" = "prod" ]; then
  dbt source freshness --select source:src_ga4 || {
      echo "STALE SOURCE: GA4 export missing/late >48h. Aborting build." >&2
      # block: do NOT build marts on stale data; page on-call, do not ship a brief
      ./ci/alert.sh --severity page --reason "ga4-source-stale"
      exit 1
  }
fi
```

On a stale source we block, we do not degrade. The pipeline aborts before any mart materializes, the previous (good) marts remain in place, and on-call is paged. The Orchestrator, seeing no fresh build, runs no diagnosis that day rather than diagnosing on a partial load - failing closed, consistent with the run's reconcile/Critic guardrails.

On the static sample, `dbt source freshness` will always report the shards as years stale, which is correct but useless. We make it informational by scoping the gate to `target.name == 'prod'` (above) and by leaving freshness *declared* but downgraded elsewhere - the check still documents the production SLA in `dbt docs` and runs as a no-op signal in dev.

7.2 Model-level freshness / build_after (dbt 1.9)

dbt 1.9 lets models declare their own freshness/ `build_after` so the scheduler rebuilds a model only when its inputs are fresh, instead of on a blind cron. We apply it to the keystone facts:

```yaml
- name: fct_daily_funnel
  config:
    freshness:
      build_after: {count: 6, period: hour}   # rebuild only if >6h since last build AND source is fresh
      updates_on: any   # rebuild when ANY upstream (events shard) advances
```

This couples the Monitor/Decompose feed (`fct_daily_funnel`) to source arrival: it rebuilds shortly after the morning export lands, not on a fixed wall-clock that might fire before the data exists. On the static sample `build_after` simply never re-triggers (no new shards), which is the correct no-op.

7.3 Per-layer freshness SLAs

Freshness propagates down the DAG with widening tolerance - staging must be as fresh as the source; marts inherit source freshness plus their build cadence; the semantic layer is only as fresh as the marts it reads.

Layer	Model(s)	Refresh cadence (prod)	Warn after	Error after	On breach	Static-sample behavior
source	`src_ga4.events_*`	daily, ~morning UTC	36h	48h	block build + page	informational no-op
staging	`stg_ga4__*` (view)	on read	inherits source	inherits source	n/a (views)	inherits source
intermediate	`int_ga4__sessionized` , `int_ga4__funnel_steps`	per build (ephemeral/view)	36h	48h	abort downstream	no-op
marts/core	`fct_funnel` , `fct_sessions` , `fct_daily_funnel`	daily incremental, ~07:00 UTC	30h	48h	hold marts; page	recency = warn
marts/finance,growth	`fct_orders` , `fct_order_items` , `fct_funnel_by_dim` , `fct_cohorts`	daily table	30h	48h	hold marts	recency = warn
dims	`dim_users/items/channels/date`	daily / on change (`snap_dim_items` SCD2)	7d	14d	warn	warn
semantic	`semantic_layer.yaml` → semantic-mcp	reads marts	= marts	= marts	refuse query if marts stale	n/a

`dim_date` is a fixed conformed spine over the dataset window, so it has no meaningful freshness SLA (it is regenerated only when the window changes).

7.4 Partition-lateness handling

GA4 occasionally back-fills a shard: yesterday's `events_YYYYMMDD` can receive late hits hours after first landing. A naive "only build today's partition" incremental would permanently miss those rows. Our incremental config absorbs this with a **3-day lookback** that re-materializes recent partitions every run:

```
{{ config(
    materialized='incremental',
    incremental_strategy='insert_overwrite',
    partition_by={'field': 'event_date', 'data_type': 'date', 'granularity': 'day'},
    cluster_by=['device_category', 'channel_group'],
    require_partition_filter=true
) }}

select ...
from {{ ref('stg_ga4__events') }}
{% if is_incremental() %}
  -- reprocess the trailing 3 days so late-arriving shards correct prior partitions
  where event_date ≥ date_sub(current_date(), interval 3 day)
{% endif %}
```

`insert_overwrite` atomically replaces those 3 day-partitions, so late hits land and any earlier double-count is overwritten, not appended. Three days covers GA4's typical back-fill window with margin; a shard that arrives >3 days late is rare and is caught instead by the recency test (§7.5) flagging a gap. On the static sample `is_incremental()` is false on first build and the lookback predicate is simply never exercised.

7.5 Recency tests

Freshness (a `dbt source freshness` concept) checks *source* lag; recency tests assert that the *built marts* actually contain a recent row - catching the case where the source was fresh but the build silently skipped a partition.

```
- name: fct_daily_funnel
  tests:
    - dbt_utils.recency:
        datepart: day
        field: event_date
        interval: 2
        config:
          severity: "{{ 'error' if target.name == 'prod' else 'warn' }}"
    # row-count recency: today's partition must carry a plausible volume
    - dbt_expectations.expect_table_row_count_to_be_between:
        min_value: 1
        row_condition: "event_date = date_sub(current_date(), interval 1 day)"
        config:
          severity: "{{ 'error' if target.name == 'prod' else 'warn' }}"
```

`dbt_utils.recency` errors in prod if the newest `event_date` is more than 2 days old; the `dbt_expectations` row-count-recency test errors if yesterday's partition exists but is empty (a half-loaded shard). Both are scoped to `severity: warn` off prod, so on the **static sample they degrade to warnings** - the analyst sees an informational "newest partition is 2021-01-31, expected within 2 days" notice without the build failing. This is the deliberate graceful-degradation contract: the production SLAs are fully declared and visible in `dbt docs`, run as hard gates in prod, and become benign informational signals on the historical public dataset.